

Manual por sesiones

Curso Playwright con JavaScript

Documento unico para estudiar, reutilizar comandos, entender archivos y repasar cada sesion del curso.

Proyecto: C:\Users\User\Documents\PLAYWRIGHT_JAVASCRIPT\Curso

Generado desde MANUAL_SESIONES_CURSO_PLAYWRIGHT.md

Manual por sesiones - Curso Playwright con JavaScript

Este documento resume y organiza las sesiones del curso que se trabajaron en el proyecto local:

C:\Users\User\Documents\PLAYWRIGHT_JAVASCRIPT\Curso

La idea de este manual es que puedas volver a cada sesion rapidamente y entender:

- De que trato la clase.
- Para que sirvio.
- Que mostro el profesor.
- Que archivos se crearon o modificaron.
- Que comandos se usaron.
- Por que es importante aprender ese tema.
- Como puedes reutilizarlo en tus propios proyectos.

Importante: Algunas pruebas usan el dominio <https://testingqarvn.com.es/>. En esta maquina ese dominio no resuelve DNS actualmente, asi que esas pruebas pueden fallar aunque el codigo este bien.

Vista general del curso trabajado

Sesion	Video	Tema principal	Resultado principal
1	Video 1.mkv	Primeros tests, Codegen, inputs y radio buttons	Proyecto Playwright con pruebas iniciales
2	Video 2.mkv	Archivo <code>playwright.config.js</code>	Configuracion del runner, reportes, screenshots y video
3	video 3.mkv	Introduccion a Page Object Model	Clase FJ y primeros archivos POM
4	Video 4.mkv	Validaciones, expect, try/catch	Funciones reutilizables de validacion
5	Video 5.mkv	Combos, selects y acciones de mouse	Archivo <code>Actions.spec.js</code> y metodos para UI
6	Video 6.mkv	Drag/drop, copiar/pegar, upload, calendario y paralelizacion	Acciones avanzadas y <code>Paralelo.spec.js</code>
7	Video 7.mkv	Parametros, <code>.env</code> y Excel	<code>Parametros.spec.js</code> , <code>Data.xlsx</code> , <code>dotenv</code> , <code>xlsx</code>
8	Video 8.mkv	Jenkins, NodeJS, Git y GitHub	Preparacion para integracion continua

Sesion 1 - Primeros ejercicios de Playwright

De que trata esta sesion

En esta sesion se inicia el proyecto con Playwright y JavaScript. El profesor muestra pruebas basicas generadas o escritas directamente, incluyendo:

- Pruebas base de Playwright.
- Uso de Codegen.
- Escritura en campos de texto.
- Uso de `fill`, `type` y `press`.
- Uso de `test.only`.
- Radio buttons.

Para que sirvio esta sesion

Sirvio para dar los primeros pasos con Playwright y entender como se escribe y ejecuta una prueba automatizada.

El objetivo no era todavia crear una estructura profesional, sino familiarizarse con:

- Archivos `.spec.js`.
- El objeto `page`.
- Selectores.
- Comandos basicos de Playwright.
- Ejecucion de pruebas desde terminal.

Que mostro el profesor

El profesor mostro como crear y ejecutar pruebas sencillas, por ejemplo:

- Abrir una pagina.
- Escribir en un input.
- Presionar teclas.
- Seleccionar radio buttons.
- Ejecutar una prueba especifica.
- Usar `test.only` para correr solo una prueba mientras se aprende.

Archivos creados o usados

```
Curso/tests/Codegen.spec.js
Curso/tests/Demo2.spec.js
Curso/tests/example.spec.js
Curso/tests/Input1.spec.js
Curso/tests/Input2.spec.js
Curso/tests/Radio.spec.js
```

Comandos usados

```
npm.cmd run test:demo -- --workers=1
```

Sirve para ejecutar el script `test:demo` definido en `package.json`.

```
npm.cmd run test:input -- --workers=1
```

Sirve para ejecutar pruebas relacionadas con inputs.

```
npm.cmd run test:radio -- --workers=1
```

Sirve para ejecutar la prueba de radio buttons.

```
npx.cmd playwright test Codegen.spec.js --workers=1
```

Ejecuta directamente el archivo `Codegen.spec.js`.

```
npx.cmd playwright test Input1.spec.js --workers=1
```

Ejecuta directamente el archivo `Input1.spec.js`.

```
npx.cmd playwright test example.spec.js --workers=1
```

Ejecuta el ejemplo base generado por Playwright.

```
npm.cmd test -- --list
```

Muestra la lista de pruebas detectadas por Playwright.

Por que es importante aprender esto

Esta sesion es importante porque muestra la base de todo el curso. Antes de usar Page Object Model, Excel, Jenkins o paralelizacion, necesitas entender como Playwright abre una pagina y ejecuta acciones simples.

Como puedes reutilizarlo

Puedes usar estos archivos como referencia cuando quieras:

- Crear tu primer test.
- Probar si Playwright esta instalado.
- Practicar selectores.
- Automatizar formularios simples.
- Ejecutar un solo archivo de prueba.

Sesion 2 - Archivo de configuracion `playwright.config.js`

De que trata esta sesion

Esta sesion se enfoca en el archivo de configuracion de Playwright:

```
Curso/playwright.config.js
```

El profesor explica opciones como:

- Carpeta donde viven las pruebas.
- Timeout general.
- Timeout de `expect`.
- Ejecucion en paralelo.
- Reintentos.
- Workers.

- Reporte HTML.
- Navegador.
- Modo visible o headless.
- Viewport.
- Screenshots.
- Videos.
- Trace.

Para que sirvio esta sesion

Sirvio para entender que Playwright no depende solo del codigo de los tests. Tambien tiene un archivo central que controla como se ejecutan las pruebas.

Que mostro el profesor

El profesor mostro como configurar:

```
testDir: './tests'
```

Indica que Playwright debe buscar pruebas dentro de la carpeta tests.

```
headless: false
```

Hace que el navegador se vea mientras corre la prueba.

```
video: 'on'
```

Graba video de las pruebas.

```
screenshot: 'only-on-failure'
```

Toma captura cuando una prueba falla.

```
trace: 'on-first-retry'
```

Guarda trazas para investigar errores.

Archivos creados o modificados

```
Curso/playwright.config.js
Curso/playwright.config_base.js
Curso/package.json
```

Comandos usados

```
npx.cmd playwright test
```

Ejecuta las pruebas usando la configuracion de playwright.config.js.

```
npx.cmd playwright show-report
```

Abre el reporte HTML generado por Playwright.

```
npm.cmd test -- --list
```

Verifica que Playwright detecta correctamente los archivos .spec.js.

Por que es importante aprender esto

Porque en proyectos reales no basta con escribir pruebas. Tambien necesitas decidir:

- Si se ejecutan en paralelo.
- Si se graban videos.
- Si se guardan trazas.
- Cuanto tiempo esperar antes de fallar.
- Que navegador usar.

Como puedes reutilizarlo

Puedes copiar esta configuracion como base para otros proyectos Playwright. Luego puedes ajustar:

- `headless` a `true` si quieres correr sin ver el navegador.
- `video` a `retain-on-failure` para ahorrar espacio.
- `workers` para controlar la paralelizacion.
- `viewport` para simular otro tamaño de pantalla.

Sesion 3 - Introduccion a Page Object Model

De que trata esta sesion

En esta sesion el profesor introduce Page Object Model, conocido como POM.

La idea principal es dejar de repetir código en cada prueba y empezar a crear funciones reutilizables.

Para que sirvio esta sesion

Sirvio para crear una clase reutilizable llamada FJ.

Esa clase guarda metodos comunes como:

- Abrir una pagina.
- Esperar tiempo.
- Hacer scroll.
- Escribir texto.
- Hacer clic.
- Validar texto.
- Validar URL.

Que mostro el profesor

Mostro que en lugar de escribir muchas veces:

```
await page.locator(selector).fill(valor)
```

se puede crear un metodo:

```
async texto(selector, val, tiempo=10){
  await this.page.locator(selector).fill(val)
  await sleep(tiempo)
}
```

Y luego usarlo asi:

```
await f.texto("//input[contains(@id,'campo')]","Rodrigo",tp)
```

Archivos creados o usados

```
Curso/tests/Funciones_curso.js  
Curso/tests/Funciones.js  
Curso/tests/PO_UNO.spec.js  
Curso/tests/PO_DOS.spec.js
```

Comandos usados

```
node.exe --check .\tests\Funciones_curso.js
```

Verifica que el archivo JavaScript no tenga errores de sintaxis.

```
node.exe --check .\tests\PO_UNO.spec.js
```

Verifica sintaxis del primer archivo POM.

```
npm.cmd test -- --list
```

Confirma que Playwright detecta los nuevos tests.

```
npx.cmd playwright test PO_UNO.spec.js --workers=1
```

Ejecuta el archivo PO_UNO.spec.js.

Por que es importante aprender esto

Page Object Model es muy importante porque ayuda a mantener ordenado el proyecto. Si tienes muchos tests y repites selectores por todas partes, cualquier cambio en la pagina te obliga a modificar muchos archivos.

Con POM puedes centralizar funciones y reutilizarlas.

Como puedes reutilizarlo

Puedes reutilizar Funciones_curso.js como base para tus propios proyectos. Cada vez que repitas una accion, puedes convertirla en metodo.

Ejemplo:

```
await f.openURL("https://demoqa.com/buttons")  
await f.click("//button[text()='Click Me']")
```

Sesion 4 - Validaciones, expect y manejo de errores

De que trata esta sesion

Esta sesion continua con Page Object Model, pero ahora agrega validaciones y manejo de errores.

El profesor introduce:

- expect.
- expect.soft.
- try/catch.
- Validacion de URL.
- Validacion de titulo.
- Lectura del valor de un campo.

Para que sirvio esta sesion

Sirvio para que las pruebas no solo hagan acciones, sino que tambien validen resultados.

Una prueba automatizada no debe limitarse a llenar formularios. Tambien debe comprobar que la pagina respondio correctamente.

Que mostro el profesor

Agrego metodos como:

```
async validar_titulo(titulo, tiempo=10){
  await expect(this.page).toHaveTitle(titulo);
  await sleep(tiempo)
}
```

Y tambien manejo de errores:

```
async texto_try(selector, val, tiempo=10){
  try{
    const locator = this.page.locator(selector);
    await expect(locator).toBeVisible();
    await expect(locator).toBeEnabled();
    await expect(locator).toBeEmpty();
    await this.page.locator(selector).fill(val)
    await sleep(tiempo)
  }catch(error){
    console.log("Campo con algun Error")
  }
}
```

Archivos modificados

```
Curso/tests/Funciones_curso.js
Curso/tests/P0_DOS.spec.js
```

Comandos usados

```
node.exe --check .\tests\Funciones_curso.js
```

Verifica sintaxis de la clase de funciones.

```
node.exe --check .\tests\P0_DOS.spec.js
```

Verifica sintaxis del test POM.

```
npm.cmd test -- --list
```

Lista las pruebas detectadas.

```
npx.cmd playwright test P0_DOS.spec.js --workers=1
```

Ejecuta el segundo test POM.

Por que es importante aprender esto

Porque una automatizacion sin validaciones puede parecer exitosa aunque no haya comprobado nada importante.

Las validaciones permiten responder preguntas como:

- La pagina cargo correctamente?
- El titulo es el esperado?
- El texto aparece?
- La URL final es correcta?

- El campo contiene el valor esperado?

Como puedes reutilizarlo

Puedes usar estos metodos en cualquier test donde necesites validar comportamiento:

```
await f.validar_texto("//h1", "Bienvenido")
await f.validar_url("https://mi-sitio.com/home")
await f.validar_titulo("Mi Pagina")
```

Sesion 5 - Combos, selects y acciones de mouse

De que trata esta sesion

Esta sesion trabaja con elementos mas avanzados de formularios e interaccion:

- Combos simples.
- Combos multiples.
- Select por valor.
- Select por label.
- Click dinamico.
- Click derecho.
- Doble click.
- Mouse over.
- Reintentos en la configuracion.

Para que sirvio esta sesion

Sirvio para automatizar controles que aparecen frecuentemente en aplicaciones reales.

No todos los formularios son solo inputs de texto. Muchas veces hay listas desplegables, botones dinamicos, menus y acciones de mouse.

Que mostro el profesor

Agrego metodos como:

```
async combo_value(selector, val, tiempo=1){
  const cam = await this.page.locator(selector)
  await cam.selectOption(val)
  await sleep(tiempo)
}
```

Y:

```
async click_dinamico(texto, tiempo=1){
  const sel = await this.page.getByRole('button', { name: texto, exact: true })
  await sel.click();
  await sleep(tiempo)
}
```

Archivos creados o modificados

```
Curso/tests/Actions.spec.js
Curso/tests/P0_TRES.spec.js
```

```
Curso/tests/Funciones_curso.js
Curso/playwright.config.js
```

Comandos usados

```
node.exe --check .\tests\Funciones_curso.js
node.exe --check .\tests\Actions.spec.js
node.exe --check .\tests\PO_TRES.spec.js
npx.cmd playwright test Actions.spec.js --workers=1
npm.cmd test -- --list
```

Por que es importante aprender esto

Porque las aplicaciones reales tienen muchos tipos de controles. Si solo sabes escribir en inputs, tus pruebas seran muy limitadas.

Esta sesion te prepara para automatizar:

- Formularios con selects.
- Botones que no tienen un ID fijo.
- Acciones especiales de mouse.
- Menus que aparecen al pasar el cursor.

Como puedes reutilizarlo

Puedes reutilizar `Actions.spec.js` como referencia para automatizar paginas con controles complejos.

Ejemplo:

```
await f.combo_value("#cars", "volvo", tp)
await f.click_dinamico("Click Me", tp)
await f.mouse_over("//a[contains(., 'Node.js')]", tp)
```

Sesion 6 - Acciones avanzadas y paralelizacion

De que trata esta sesion

Esta sesion agrega acciones mas avanzadas:

- Drag and drop.
- Copiar texto.
- Pegar texto.
- Copiar desde inputs.
- Subir archivos.
- Limpiar archivos cargados.
- Manejar calendarios.
- Ejecutar muchas pruebas con un ciclo for.

Para que sirvio esta sesion

Sirvio para trabajar casos mas cercanos a proyectos reales. Muchas aplicaciones tienen carga de archivos, calendarios o acciones de arrastrar y soltar.

Tambien introduce la idea de generar multiples pruebas automaticamente.

Que mostro el profesor

Agrego metodos como:

```
async Upload_file(selector, archivo, tiempo=1000){
  const sel = await this.page.locator(selector)
  await sel.setInputFiles(archivo);
  await sleep(tiempo)
}
```

Y creo un archivo de pruebas paralelas:

Curso/tests/Paralelo.spec.js

Ese archivo genera pruebas con:

```
for(x=1;x<=50;x++){
  test(`Combo numero_${x}`, async ({ page }) => {
    ...
  });
}
```

Archivos creados o modificados

Curso/tests/Actions.spec.js
Curso/tests/Funciones_curso.js
Curso/tests/Paralelo.spec.js
Curso/tests/archivos/CURSOS.txt

Comandos usados

```
node.exe --check .\tests\Funciones_curso.js
node.exe --check .\tests\Actions.spec.js
node.exe --check .\tests\Paralelo.spec.js
npx.cmd playwright test Actions.spec.js -g "Drag and Drop" --workers=1 --retries=0
```

Ejecuta solo la prueba cuyo nombre coincide con Drag and Drop.

```
npm.cmd test -- --list
```

Muestra todas las pruebas detectadas.

Por que es importante aprender esto

Porque automatizar aplicaciones reales exige mas que llenar formularios. Esta sesion enseña acciones que aparecen mucho en sistemas empresariales:

- Cargar documentos.
- Copiar datos.
- Seleccionar fechas.
- Arrastrar elementos.
- Ejecutar varias pruebas similares.

Como puedes reutilizarlo

Puedes tomar los metodos de `Funciones_curso.js` y usarlos en otros tests:

```
await f.Upload_file("//input[@type='file']", "tests/archivos/CURSOS.txt", tp)
await f.drag_drop("//div[@id='origen']", "//div[@id='destino']", tp)
await f.Tab("//input[contains(@id, 'fecha')]", tp)
```

Nota importante sobre cambios actuales

DemoQA cambio el texto visible de la zona de drop. Por eso se ajusto internamente `drag_drop` para seguir funcionando con la version actual de la pagina.

Sesion 7 - Parametros, variables de ambiente y Excel

De que trata esta sesion

Esta sesion enseña a no dejar los datos quemados directamente en el test.

El profesor trabaja con:

- Arrays de datos.
- Variables de ambiente en `.env`.
- Paquete `dotenv`.
- Paquete `xlsx`.
- Archivo Excel `Data.xlsx`.
- Lectura de filas desde Excel.
- Uso de `test.only` en `Data Excel`.

Para que sirvio esta sesion

Sirvio para parametrizar pruebas. Es decir, usar datos externos para ejecutar el mismo flujo con diferentes valores.

Esto es muy importante porque en automatizacion real no quieres crear un test completo por cada usuario o por cada registro.

Que mostro el profesor

Primero uso un array:

```
const datos = ['Rodrigo', 'Villanueva', 'rod@gmail.com', '34545', 'Direccion uno', 'Java'];
```

Despues uso `.env`:

```
NOMBRE=Juan
APELLIDO=Perez
EMAIL=juan@gmail.com
TELEFONO=786876
DIRECCION=Direccion uno
```

Y finalmente Excel:

```
var XL=require("xlsx");
var libro=XL.readFile("Data.xlsx");
let datos_xl=libro.SheetNames;
const hoja=datos_xl[0]
const excel=XL.utils.sheet_to_json(libro.Sheets[hoja])
```

Archivos creados o modificados

```
Curso/tests/Parametros.spec.js
Curso/.env
Curso/Data.xlsx
Curso/package.json
Curso/package-lock.json
Curso/playwright.config.js
```

Dependencias instaladas

```
npm.cmd install dotenv@^16.0.3 xlsx@^0.18.5
```

dotenv permite leer variables desde .env.

xlsx permite leer archivos Excel.

Comandos usados

```
npm.cmd install dotenv@^16.0.3 xlsx@^0.18.5
```

Instala las dependencias necesarias para parametros y Excel.

```
npx.cmd playwright test Parametros.spec.js --list
```

Lista las pruebas dentro de Parametros.spec.js.

```
npm.cmd test -- --list
```

Lista todas las pruebas del proyecto.

```
npx.cmd playwright test Parametros.spec.js -g "Data Excel" --workers=1 --retries=0
```

Ejecuta solo la prueba Data Excel.

Por que es importante aprender esto

Porque las pruebas con datos externos son mucho mas poderosas.

Con Excel o .env puedes:

- Cambiar datos sin tocar el codigo.
- Probar varios usuarios.
- Separar configuracion de logica.
- Reutilizar el mismo flujo con muchas entradas.
- Preparar el proyecto para ejecuciones en Jenkins.

Como puedes reutilizarlo

Puedes crear tus propios archivos Excel con columnas como:

```
nombre | apellido | email | tel | direccion
```

Y luego recorrerlos:

```
for(const fila of excel){
  await f.texto_val("//input[contains(@id, 'wsf-1-field-29')]", fila['nombre'], tp)
}
```

Nota de seguridad

No debes guardar contraseñas reales, tokens o claves privadas dentro de `.env` si luego vas a subir el proyecto a GitHub.

Sesion 8 - Jenkins, NodeJS, Git y GitHub

De que trata esta sesion

Esta sesion cambia de tema: ya no se agregan pruebas nuevas de Playwright. El profesor empieza a preparar el entorno para integracion continua.

Los temas principales son:

- Jenkins.
- Configuracion inicial de Jenkins.
- Plugins sugeridos.
- Usuario administrador.
- Configuracion global de herramientas.
- NodeJS dentro de Jenkins.
- Git.
- GitHub.
- `git config`.
- `git init`.

Para que sirvio esta sesion

Sirvio para preparar el proyecto para que en el futuro pueda ejecutarse desde Jenkins y conectarse con GitHub.

En proyectos reales, las pruebas no se ejecutan solo manualmente. Tambien pueden ejecutarse automaticamente:

- Cada vez que se sube codigo.
- Cada cierto horario.
- Antes de aprobar cambios.
- En un servidor de integracion continua.

Que mostro el profesor

El profesor mostro la pantalla de Jenkins y la configuracion de NodeJS:

```
Manage Jenkins
Global Tool Configuration
NodeJS
Name: NODE_JS
Install automatically
Version: NodeJS 19.6.0
```

Tambien mostro comandos Git:

```
git --version
```

Sirve para verificar que Git esta instalado.

```
git config --global user.name "Rodrigo"
```

Configura el nombre global de Git.

```
git config --global user.email "rodrigodrupal1@gmail.com"
```

Configura el correo global de Git.

```
git init
```

Inicializa un repositorio Git en la carpeta actual.

Archivos creados o modificados

En nuestro proyecto se creo:

```
Curso/Jenkins_Git_Video8.md
```

Ese archivo documenta lo mostrado por el profesor sin cambiar la configuracion global de tu maquina.

Comandos usados o verificados localmente

```
git --version
```

Resultado local:

```
git version 2.52.0.windows.1
```

```
node --version
```

Resultado local:

```
v22.20.0
```

```
npm.cmd --version
```

Resultado local:

```
10.9.3
```

```
npm.cmd test -- --list
```

Confirma que el proyecto Playwright sigue detectando las pruebas.

Por que es importante aprender esto

Porque Git, GitHub y Jenkins permiten llevar el proyecto a un nivel mas profesional.

Aprender esto sirve para:

- Guardar versiones del codigo.
- Subir el proyecto a GitHub.
- Compartirlo con otros.
- Ejecutar pruebas automaticamente.
- Integrar Playwright en pipelines de CI/CD.

Como puedes reutilizarlo

Cuando quieras preparar este proyecto para GitHub, puedes hacer:

```
cd C:\Users\User\Documents\PLAYWRIGHT_JAVASCRIPT\Curso
git init
```

Luego, con tus propios datos:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@example.com"
```

Importante: No se aplicaron los datos del profesor en tu configuracion global porque eso cambiaria la identidad Git de tu equipo.

Tabla rapida de comandos del curso

Comando	Para que sirve	Donde se ejecuta
<code>npm.cmd test -- --list</code>	Lista las pruebas detectadas	Curso
<code>npx.cmd playwright test</code>	Ejecuta pruebas Playwright	Curso
<code>npx.cmd playwright show-report</code>	Abre reporte HTML	Curso
<code>node.exe --check archivo.js</code>	Verifica sintaxis JavaScript	Curso
<code>npm.cmd install dotenv@^16.0.3 xlsx@^0.18.5</code>	Instala lectura de .env y Excel	Curso
<code>npx.cmd playwright test Parametros.spec.js --list</code>	Lista tests de parametros	Curso
<code>npx.cmd playwright test Actions.spec.js -g "Drag and Drop"</code>	Ejecuta una prueba especifica	Curso
<code>git --version</code>	Verifica Git instalado	Cualquier carpeta
<code>node --version</code>	Verifica Node instalado	Cualquier carpeta
<code>npm.cmd --version</code>	Verifica npm en Windows	Cualquier carpeta
<code>git init</code>	Inicializa repositorio Git	Carpeta del proyecto

Tabla rapida de archivos importantes

Archivo	Para que sirve
Curso/playwright.config.js	Configura como corre Playwright
Curso/package.json	Guarda scripts y dependencias
Curso/tests/Funciones_curso.js	Clase reutilizable con metodos del curso
Curso/tests/Actions.spec.js	Pruebas de combos, acciones, upload, calendario
Curso/tests/Paralelo.spec.js	Pruebas generadas con ciclo for
Curso/tests/Parametros.spec.js	Pruebas con array, .env y Excel
Curso/.env	Variables de ambiente
Curso/Data.xlsx	Datos para pruebas desde Excel
Curso/tests/archivos/CURSOS.txt	Archivo usado para prueba de upload

Archivo	Para que sirve
Curso/Jenkins_Git_Video8.md	Guia del video de Jenkins/Git

Estado actual del proyecto

Playwright detecta:

76 pruebas en 12 archivos

Hay pruebas que no pueden ejecutarse completas porque dependen de:

<https://testingqarvn.com.es/>

Ese dominio no resuelve actualmente desde esta maquina.

Como usar este manual

Si quieres estudiar el curso:

1. Lee la sesion en orden.
2. Mira que archivo se trabajo.
3. Abre ese archivo en VS Code.
4. Revisa los comandos.
5. Ejecuta solo el comando de esa sesion.
6. Si algo falla, revisa la nota de errores o dependencias externas.

Si quieres reutilizar codigo:

1. Empieza por `Funciones_curso.js`.
2. Copia el metodo que necesites.
3. Revisa en que spec se usa.
4. Adapta el selector y la URL a tu proyecto.